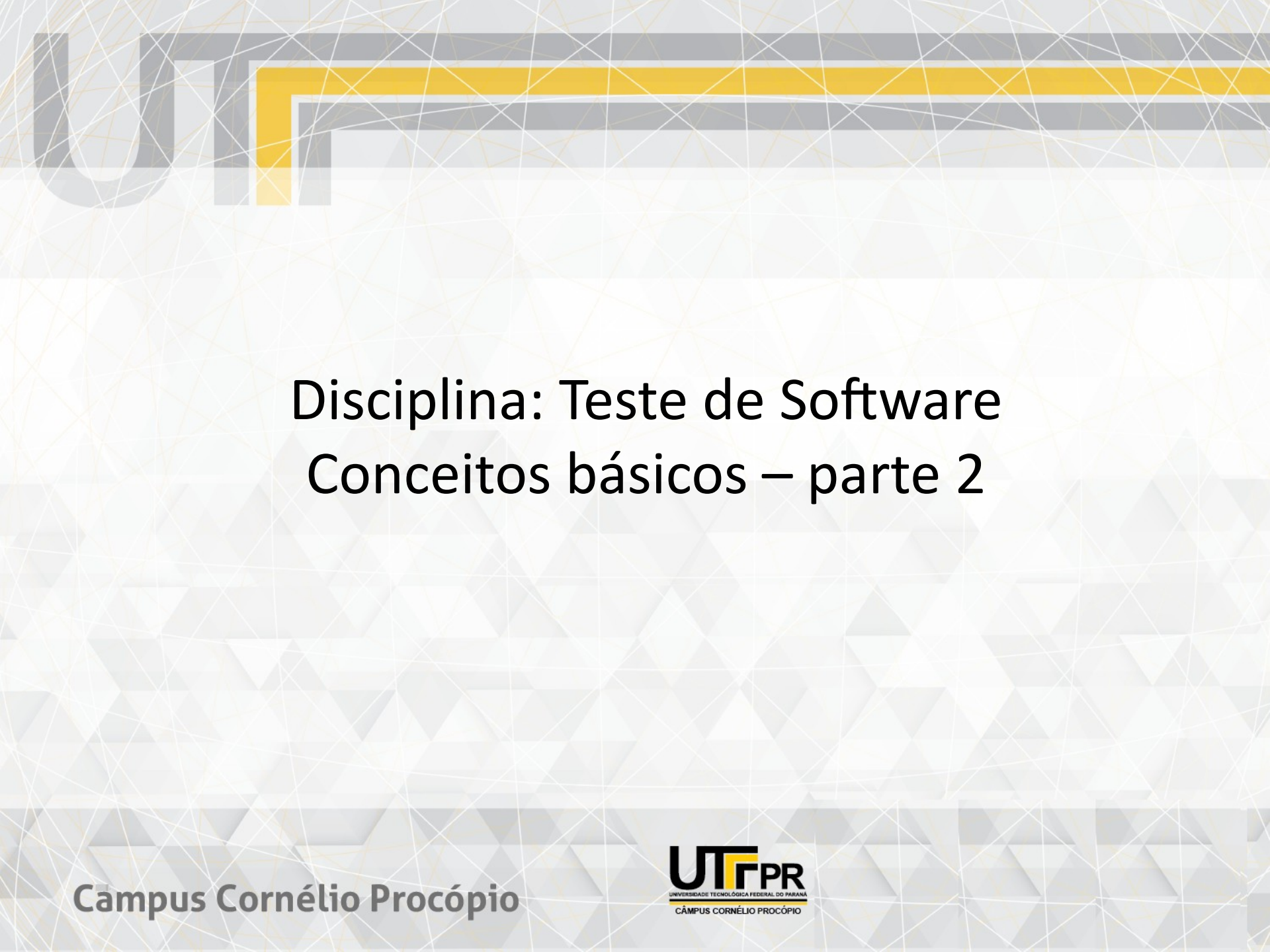




Disciplina: Teste de Software

Conceitos básicos – parte 2

Terminologia

- Critério de teste
 - Uma coleção de regras que define os requisitos de teste
 - Selecionar e avaliar o conjunto de teste. Utilizar todos os casos de teste (teste exaustivo) é inviável
 - Seleção de casos de teste representativos, que revelem falhas

Terminologia

- Critério de teste
 - Subdomínio de teste
 - Inviável testar todo o domínio
 - Exemplo: x^y , x e y são inteiros positivos

Terminologia

- Critério de teste
 - Requisito de teste
 - É um elemento específico de um artefato de software que o teste deve satisfazer ou cobrir
 - Executar uma determinada estrutura do programa ou uma funcionalidade

Terminologia

- Requisito de teste
 - O requisito de teste deve ser um comportamento mensurável e observável
 - Não há um relacionamento um para um entre um caso de uso e um requisito de teste. Normalmente, um caso de uso tem mais de um requisito de teste

Terminologia

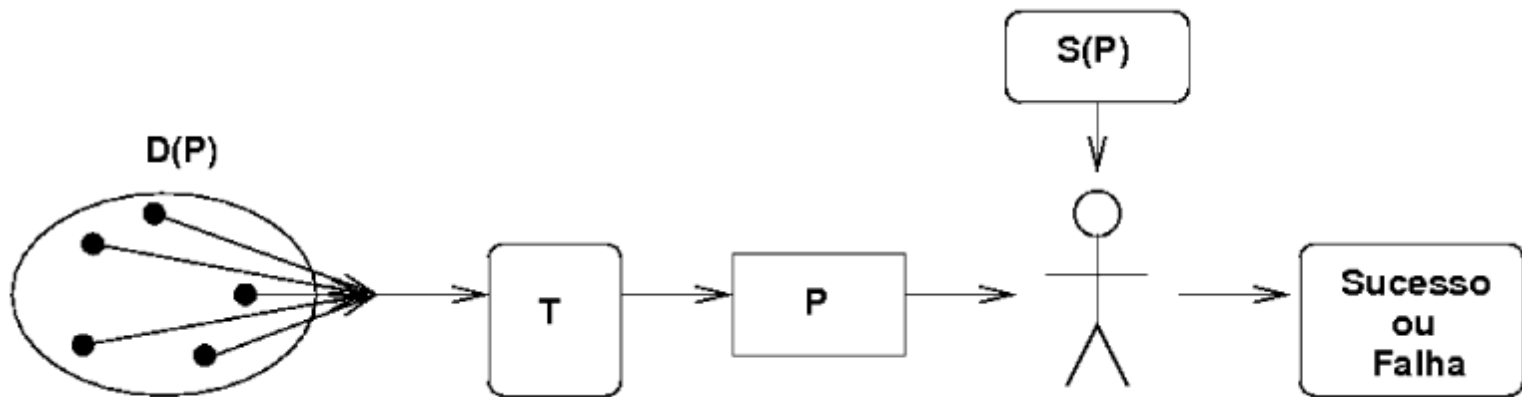
- Conjunto de teste
 - Quando um conjunto de casos de teste T satisfaz todos os requisitos de um determinado critério de teste C , T é dito adequado a C , ou simplesmente, T é *C-adequado*
 - A partir de um T que é *C-adequado* é possível obter, em teoria, infinitos conjuntos de teste *C-adequados*, incluindo mais casos de teste em T

Terminologia

- Procedimento de teste
 - Descrição dos passos necessários para realizar um conjunto de teste
 - Planejamento da atividade de teste
 - Projeto (casos de teste)
 - Execução dos testes
 - Análise dos resultados

Terminologia

- Cenário típico da atividade de teste de software



Terminologia

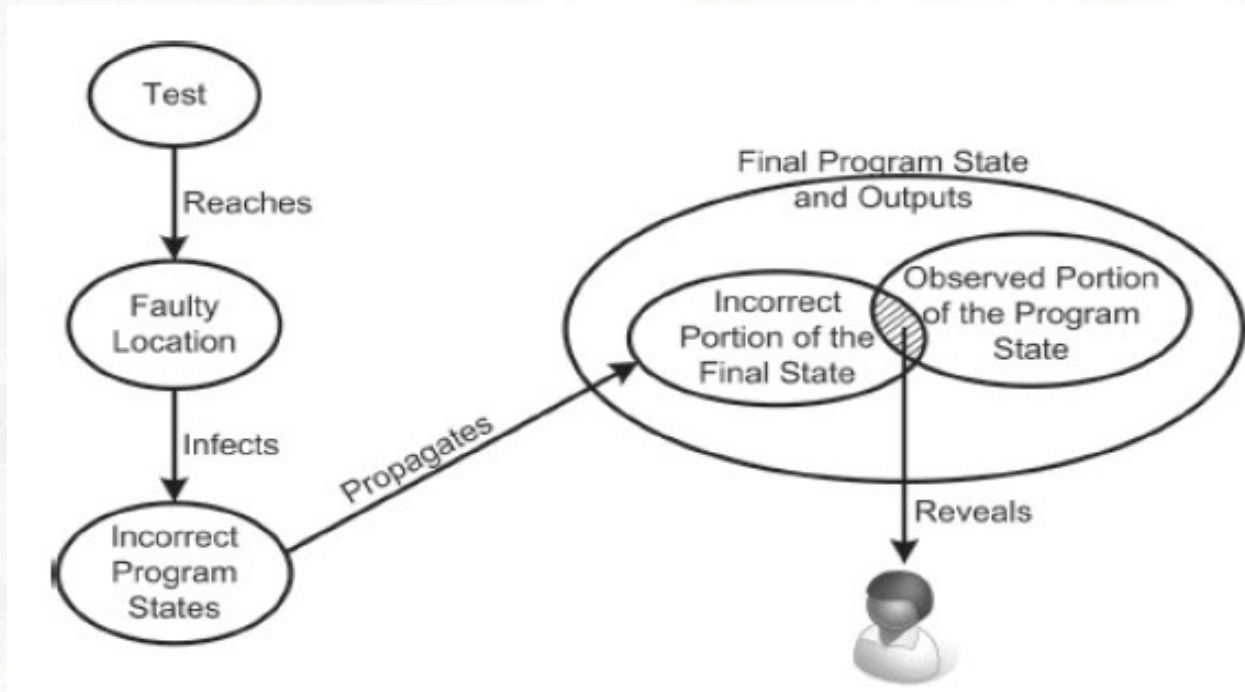
- Teste estático
 - Teste realizado sem executar o software (utilização de modelos, por exemplo)
- Teste dinâmico
 - Teste realizado com a execução do software

Terminologia

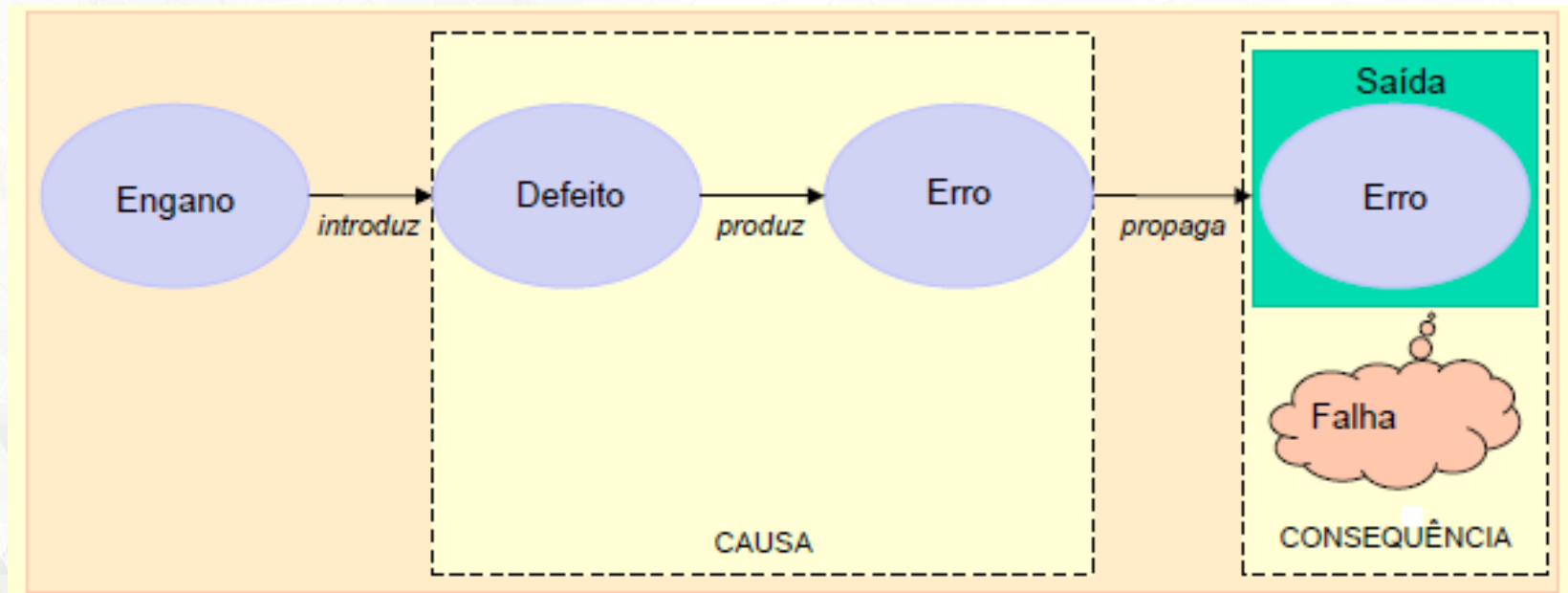
- RIPR [Ammann e Offut, 2017]
 - *Reachability*: o local ou locais no programa que contém/contêm o(s) defeito(s) deve(m) ser atingido(s)
 - *Infection*: o estado do programa deve ficar incorreto
 - *Propagation*: o estado infectado deve se propagar para causar alguma saída incorreta do programa
 - *Revealability*: é possível observar parte da porção incorreta do estado do programa ao final da execução

Terminologia

- RIPR [Ammann e Offut, 2017]



Terminologia



Terminologia

- Testabilidade
 - Observabilidade: variáveis devem ser visíveis ou consultáveis durante a execução do software - observar o comportamento do software considerando as variáveis
 - Controlabilidade: variáveis devem poder ser controladas diretamente

Terminologia

- Teste de software: encontrar critérios de teste para derivar o conjunto de casos de teste, que permitirá a revelação de falhas
- Depuração: encontrar a causa da falha e realizar a correção, isto é, encontrar o defeito considerando uma falha revelada e corrigir o defeito

Dúvidas

Técnicas de teste

- Critérios para especificação dos casos de teste
- Teste Funcional (caixa-preta)
- Teste Estrutural (caixa-branca)
- Teste baseado em defeitos

Testes funcionais

- Teste caixa-preta
- Utilização de entradas e saídas esperadas
- Comparação das saídas correntes com as saídas esperadas
- Dependência da especificação do produto de software – requisitos
- Critérios
 - Particionamento por Classes de Equivalência
 - Análise do Valor Limite
 - Grafo de causa e efeito

Testes funcionais

- Passos básicos para aplicar um critério de teste funcional
 - A especificação de requisitos é analisada
 - Entradas válidas são escolhidas para determinar se o produto em teste comporta-se corretamente
 - Entradas inválidas são escolhidas para verificar se estas são detectadas e manipuladas adequadamente

Testes funcionais

- Passos básicos para aplicar um critério de teste funcional (continuação)
 - Os casos de testes são construídos (saídas são determinadas para cada entrada)
 - O conjunto de teste é executado e as saídas obtidas são comparadas com as saídas esperadas
 - Um relatório é gerado para avaliar o resultado dos testes

Dúvidas

Referências

- Ammann, P.; Offutt, J. Introduction to software testing. Cambridge University Press, 2008.
- Beizer, B. Software testing techniques. 2nd ed. New York: Van Nostrand Reinhold Company, 1990.
- Binder, R. V. Testing object-oriented systems: Models, patterns, and tools, v. 1. Addison Wesley Longman, Inc., 1999.
- Boehm, B.; Basili, V. R. Software defect reduction top 10 list. Computer, v. 34, n. 1, p. 135–137, 2001.
- Boehm, B. W. Software engineering economics. 1st ed. Upper Saddle River, NJ, USA: Prentice Hall PTR, 1981.
- Boehm, B. W. Industrial software metrics top 10 list. IEEE Software, v. 4, n. 5, p. 84–85, 1987.
- Chow, T. S. Testing software design modelled by finite-state machines. IEEE Transactions on Software Engineering, v. 4, n. 3, p. 178–187, 1978.
- Copeland, L. A practitioner's guide to software test design. Artech House Publishers, 2004.
- Craig, R. D.; Jaskiel, S. P. Systematic software testing. Artech House Publishers, 2002.
- Dijkstra, E. W. Notes on structured programming. Relatório Técnico 70-WSK-03, Technological University of Eindhoven, The Netherlands, available at: <http://www.cs.utexas.edu/users/EWD/ewd02xx/EWD249.PDF>. Accessed on: 10-27-2007., 1970.
- Hetzel, B. The complete guide to software testing. 2nd ed. Wellesley, MA, USA: QED Information Sciences, Inc., 1988.

Referências

- IBM Orthogonal defect classification v 5.2 for software design and code. Página Web, disponível em: http://researcher.watson.ibm.com/researcher/view_project.php?id=480. 2013, Acesso em: 02/02/2014.
- ISO/IEC Quality characteristics and guidelines for their use. Padrão ISO/IEC 9126, ISO/IEC, 1991. ISO/IEC/IEEE Systems and software engineering – vocabulary. 2010.
- Maldonado, J. C.; Barbosa, E. F.; Vincenzi, A. M. R.; Delamaro, M. E.; Souza, S. R. S.; Jino, M. Introdução ao teste de software. Relatório Técnico 65 – Versão 2004-01, Instituto de Ciências Matemáticas e de Computação – ICMC-USP, disponível on-line: http://www.icmc.usp.br/CMS/Arquivos/arquivos_enviados/BIBLIOTECA_113_ND_65.pdf,2004.
- Myers, G. J. The art of software testing. Wiley, New York, 1979.
- Pezzé, M.; Young, M. Software testing and analysis: Process, principles and techniques. John Wiley & Sons, 2007.
- Roper, M. Software testing. McGrall Hill, 1994.
- Shull, F.; Basili, V.; Boehm, B.; Brown, A. W.; Costa, P.; Lindvall, M.; Port, D.; Rus, I.; Tesoriero, R.; Zelkowitz, M. What we have learned about fighting defects. In: VIII International Symposium on Software Metrics -METRICS'02, Washington, DC, USA: IEEE Computer Society, 2002, p. 249–258.